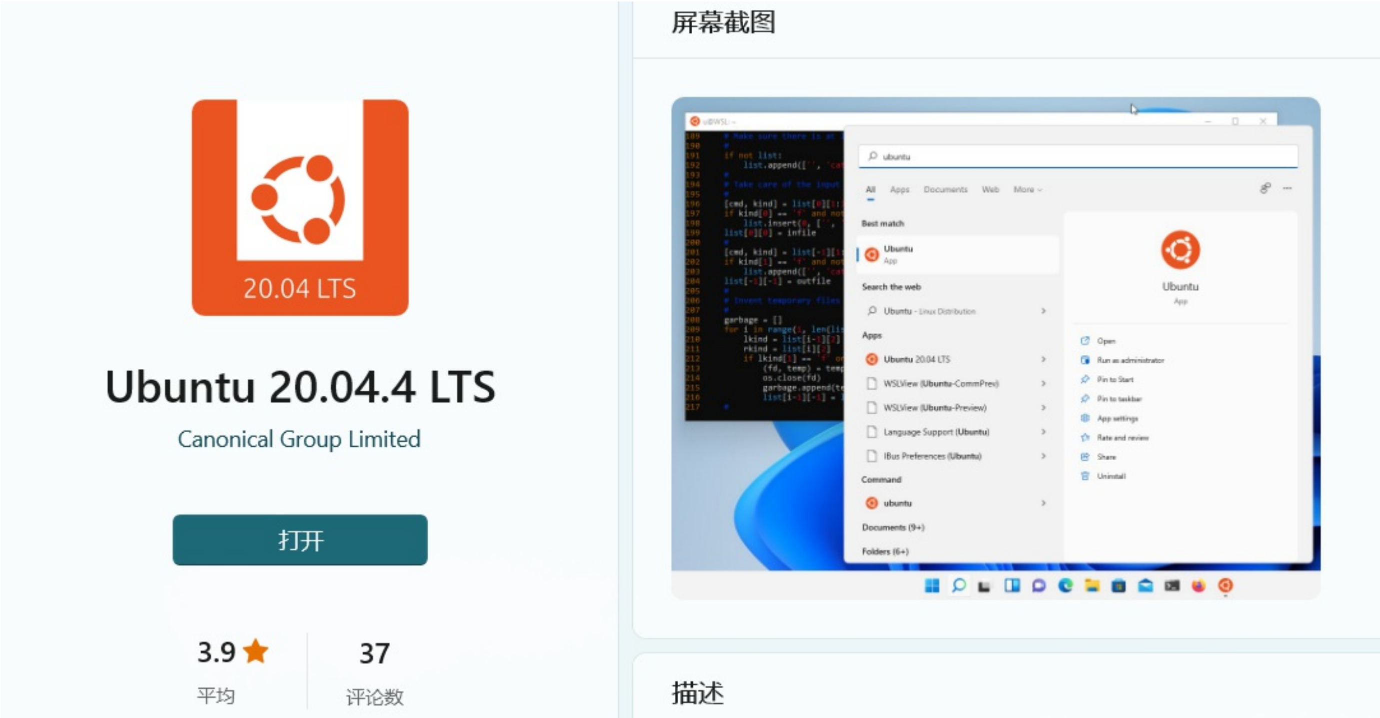
使用wsl2训练智能小车循线模型

安装wsl

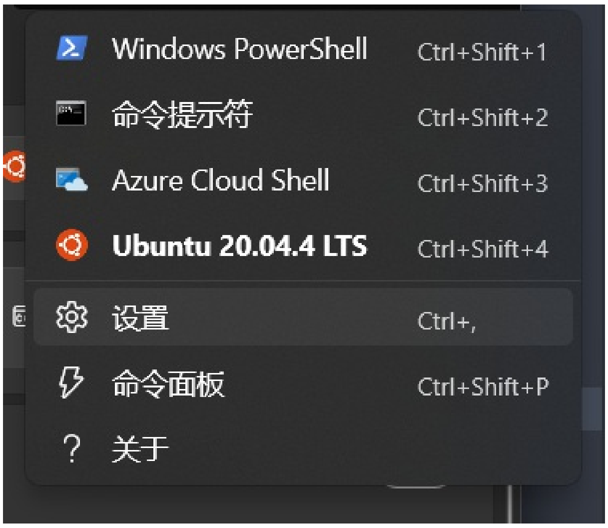
官方文档[设置wsl开发环境]()

> 安装Windows终端是可选项，强烈建议安装，使用体验会上升，以下操作都是使用Windows终端进行的。

安装完wsl后可以在Microsoft Store安装一个Ubuntu发行版，这里使用的是Ubuntu20.04.4

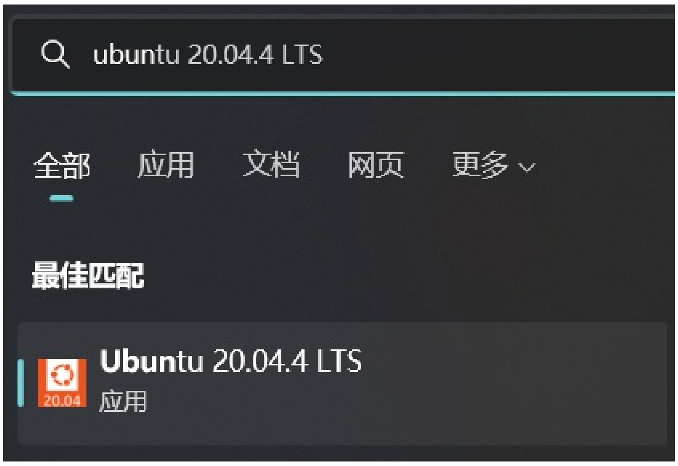


> 可选：可以在Windows终端设置默认配置



之后使用Windows终端新建一个标签页就是Ubuntu20.04.4LTS的终端了。

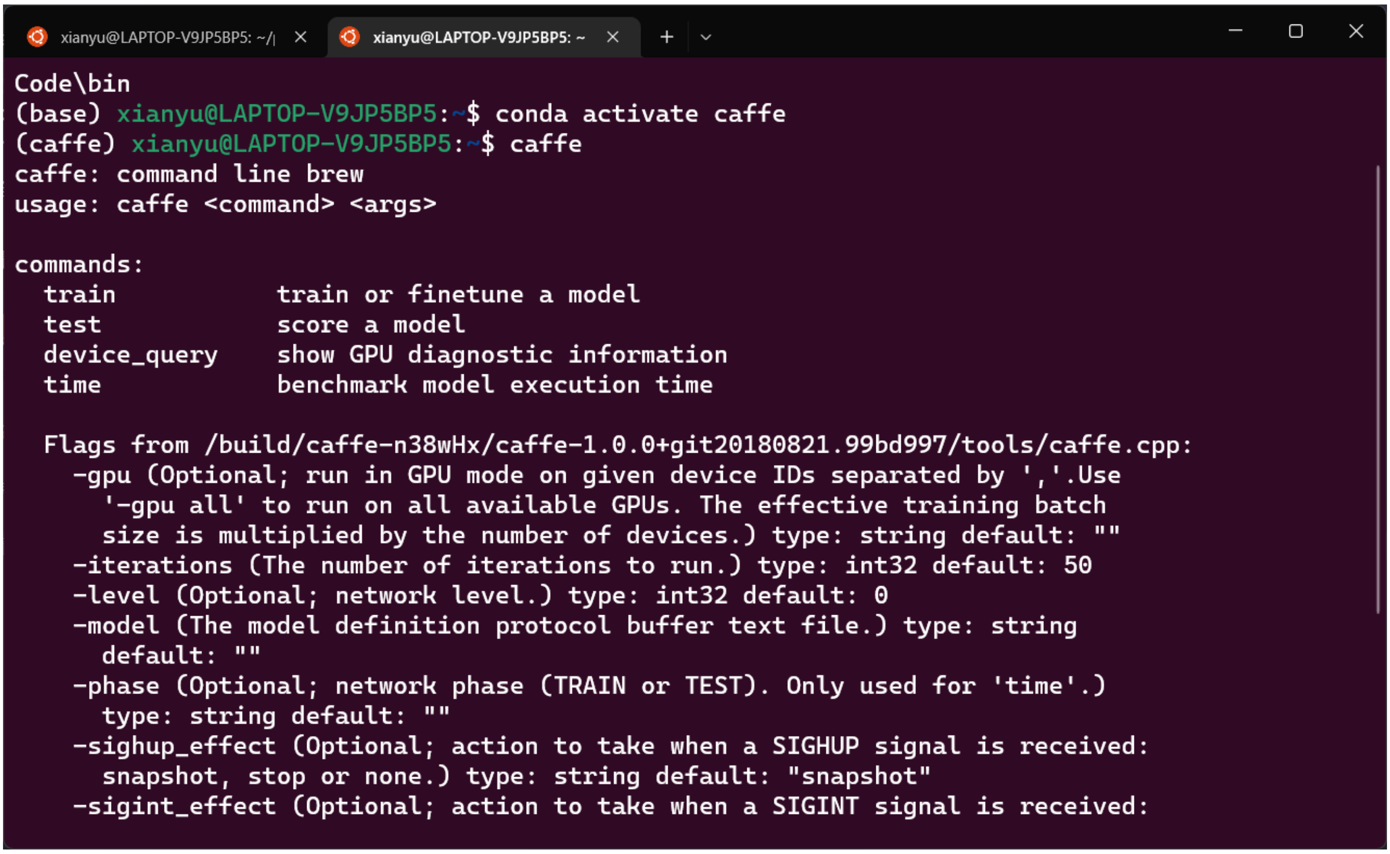
若未安装Windows终端可在开始菜单找到刚刚安装的Ubuntu，或在搜索栏直接搜索



开始配置caffe框架(cpu)

参考教程ubuntu20.04安装caffe和配置anaconda

配置好后在终端输入caffe命令，应该会出现下面的输出(反正我的是这样的)



智能小车算法开发

参考官方文档智能小车算法开发

主要根据官方文档做，这里给一些踩过的坑

- 如果ATool用不了的话可以换一台电脑，用老师给的虚拟机标注完数据，再拷贝到wsl里面
- 如果运行脚本文件`1_generate_list.py`和`2_convert_hdf5.py`出错而且怎么也解决不了，也可以用虚拟机运行完后将生成的hdf5文件夹下的内容拷贝到wsl

总的来说就是其他的步骤都可以用虚拟机完成，只是训练模型使用wsl会快很多。从下面两张图可以看出，从1:08开始到11:56就完成了5000次迭代。速度相较于虚拟机还是快了许多的。

```
I0707 01:08:38.337234   468 layer_factory.hpp:77] Creating layer data
```

```
I0707 11:56:55.917728   487 solver.cpp:327] Iteration 5000, loss = 0.00021212
I0707 11:56:55.917771   487 solver.cpp:332] Optimization Done.
I0707 11:56:55.917785   487 caffe.cpp:250] Optimization Done.
```

最重要的是训练之前需要更改一些文件里的路径，否则会报错

```
code
  Algorithm-collision_anti-drop
  Algorithm-Lane_Detection
    algorithm
      ResNet-18-Caffemodel-on-ImageNet
        deploy.prototxt
        lenet_solver_adam.prototxt
        README.md
        solver.prototxt
        train.prototxt
        train.sh
      1_generate_list.py
      2_convert_hdf5.py
      validation_regression.py
```

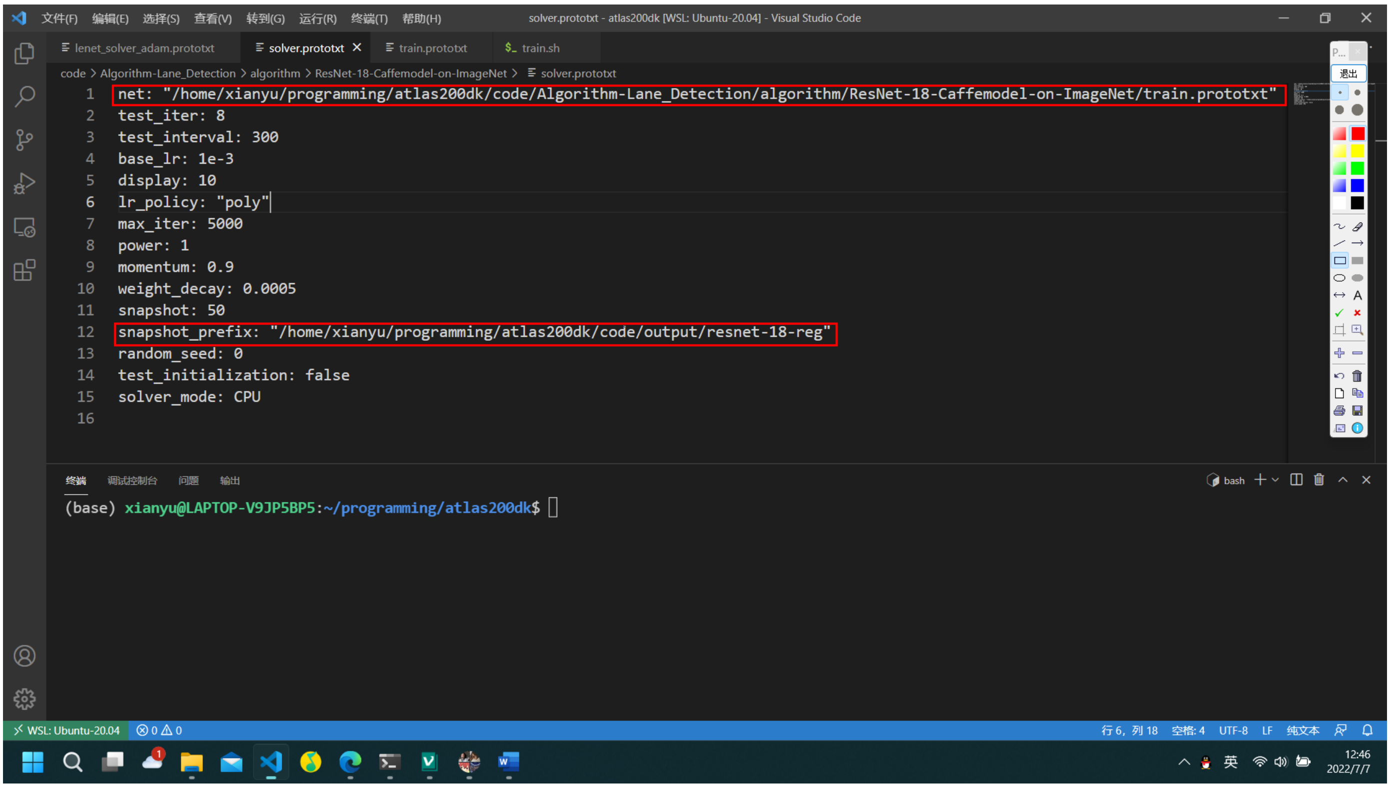
> 这个目录下的
> `train.sh`,`train.prototxt`,`solver.prototxt`,`lenet_solver_adam.prototxt`都要改
>
> code > Algorithm-Lane_Detection > algorithm > ResNet-18-Caffemodel-on-ImageNet > train.sh
>
> ```
> # export CAFFE_ROOT=$PWD
> # export PATH=$CAFFE_ROOT/build/tools:$PATH
> # export PYTHONPATH=$CAFFE_ROOT/python:$PYTHONPATH
> # export LD_LIBRARY_PATH=/home/hanxb/anaconda3/envs/caffe-ssd/lib:$LD_LIBRARY_PATH
>
> caffe train \
>   -solver lenet_solver_adam.prototxt \
>   -weights /home/xianyu/programming/atlas200dk/code/tools/ATool/model/road_following_model.caffemodel \
>   # -gpu 0
>
> ```

这里batch_size设置的是32，之前用的是128，训练的时候被kill掉了，根据电脑性能量力而行吧

code > Algorithm-Lane_Detection > algorithm > ResNet-18-Caffemodel-on-ImageNet > train.prototxt

```
name: "ResNet-18"
layer {
    name: "data"
    type: "HDF5Data"
    top: "data"
    top: "label"
    include {
        phase: TRAIN
    }
    hdf5_data_param {
        source: "/home/xianyu/programming/atlas200dk/code/tools/ATool/hdf5/train_h5.txt"
        batch_size: 32
    }
}
```

```
layer {
    name: "data"
    type: "HDF5Data"
    top: "data"
    top: "label"
    include {
        phase: TEST
    }
    hdf5_data_param {
        source: "/home/xianyu/programming/atlas200dk/code/tools/ATool/hdf5/test_h5.txt"
        batch_size: 32
    }
}
```




```
momentum: 0.9
momentum2: 0.999
# since Adam dynamically changes the learning rate, we set the base learning
# rate to a fixed value
lr_policy: "poly"
# Display every 100 iterations
display: 10
# The maximum number of iterations
max_iter: 5000
# snapshot intermediate results
snapshot: 50
snapshot_prefix: "/home/xianyu/programming/atlas200dk/code/output/resnet-18-reg-adam"
# solver mode: CPU or GPU
type: "Adam"
solver_mode: CPU
```

还有这个文件夹下的test_h5.txt,train_h5.txt

```
∨ tools
  ∨ ATool
    > build
    ∨ hdf5
        test_h5.txt
        test.h5
        train_filelist.txt
        train_h5.txt
        train.h5
        val_filelist.txt
```

code > tools > ATool > hdf5 > test_h5.txt

```
1  /home/xianyu/programming/atlas200dk/code/tools/ATool/hdf5/test.h5
2
```

code > tools > ATool > hdf5 > train_h5.txt

```
1  /home/xianyu/programming/atlas200dk/code/tools/ATool/hdf5/train.h5
2
```

到这里差不多就行了，切换到ResNet-18-Caffemodel-on-ImageNet目录下运行`train.sh`应该就可以了。

参考资料

wsl基本命令
Windows与wsl(Ubuntu)的文件互访
在Ubuntu20.04上安装anaconda